

Django architecture pattern

1. Which pattern does Django follow? a) MVVM b) MVC c) MVP d) MVT.	3
2. In Django, models are defined in which file by default? a) views.py b) models.py c) urls.py d) forms.py.	3
3. Which command is used to create migrations in Django? a) makemodel b) makemigrations c) migratecreate d) startmigration.	3
4. The Django admin interface can be customized by modifying which file? a) admin.py b) models.py c) views.py d) settings.py.	4
5. Which method is used to save a model instance in Django? a) store() b) insert() c) save() d) commit().	4
6. In Django, the render() function is used to: a) Execute queries b) Send email c) Render templates with context d) Create models.	5
7. Which HTTP method is used for updating an object in CRUD operations? a) GET b) POST c) PUT/PATCH d) DELETE. ...	5
8. In Django, uploaded files are stored using the settings variable: a) MEDIA_URL b) MEDIA_ROOT c) FILE_PATH d) STATIC_ROOT.	6
9. Which Django module handles user login and logout? a) django.auth b) django.contrib.auth c) django.sessions d) django.user.	6
10. Which Django framework is used to display temporary success or error messages? a) Toasts b) Alerts c) Messages framework d) Notifications.	7
11. Django signals are primarily used for: a) Inter-process communication b) Triggering actions on certain events c) Debugging d) Logging.	8
12. In DRF, serializers are used for: a) Database migrations b) Converting complex data to JSON/XML c) Rendering templates d) Handling sessions.	8
13. Which serializer type allows nested relationships in DRF? a) BasicSerializer b) NestedSerializer c) ModelSerializer d) HyperlinkedSerializer.	9
14. In DRF, ViewSets combine: a) Models and Templates b) Views and Routers c) CRUD operations into a single class d) Signals and Middleware.	10
15. DRF Routers are used to: a) Manage templates b) Automatically generate URL patterns for ViewSets c) Handle database queries d) Manage signals.	10
16. Which DRF class controls access to API endpoints? a) Views b) Routers c) Permissions d) Middleware.	11
17. JWT stands for: a) Java Web Tokens b) JSON Web Tokens c) JavaScript Web Transport d) JSON Writable Type.	11
18. Which DRF package is used for JWT Authentication? a) django-rest-framework-jwt b) django-jwt-auth c) rest_framework_simplejwt d) jwt-drf.	12
19. The default HTTP method allowed in DRF ViewSets is: a) GET b) POST c) GET, POST, PUT, DELETE d) Only GET and POST.	13
20. In DRF, the is_authenticated attribute belongs to: a) request.user b) request.session c) serializer.data d) view.router.	13
21. In Django, which signal is triggered after a model instance is saved? a) pre_save b) post_save c) save_signal d) after_save.	14
22. In DRF, which permission class grants read-only access to unauthenticated users but write access to authenticated users? a) IsAuthenticated b) IsAdminUser c) AllowAny d) IsAuthenticatedOrReadOnly.	14
23. Which file in an Odoo module defines the basic metadata (name, version, dependencies)? a) __init__.py b) __manifest__.py c) models.py d) views.xml.	15
24. Which relational field in Odoo defines a many-to-one relationship? a) Many2many b) Many2one c) One2many d) Char.	16
25. Which Odoo view type is used to display data in a tabular format? a) Kanban b) Form c) Tree d) Calendar.	16
26. Which decorator is used in Odoo to compute field values dynamically? a) @onchange b) @compute c) @depends d) @model.	17
27. In QWeb templates, which symbol is used for dynamic value rendering? a) {% %} b) [[]] c) t-esc d) <? ?>.	18
28. Which relational field is used when a record can have multiple associated records in both directions? a) Many2one b) One2many c) Many2many d) Char.	18
29. Which of the following is true about @api.onchange in Odoo? a) It is triggered when the record is saved b) It is triggered immediately after a field value changes c) It computes values automatically on all records d) It replaces @compute decorator.	19
30. In Odoo, which file is mandatory to load views? a) controllers.py b) __init__.py c) XML file inside views/ folder d) README file.	19
31. What does MVC stand for in Django context?. Easy Answer.	20

32. Which command applies migrations in Django?. Easy Answer.	20
33. What is the role of ModelSerializer in DRF?. Easy Answer.	21
34. Name two built-in authentication classes in DRF.. Easy Answer.	21
35. What is the default port for Django's development server?. Easy Answer.	21
36. What is the purpose of the __manifest__.py file in an Odoo module?. Easy Answer.	22
37. Differentiate between Many2one and One2many relational fields.. Easy Answer.	22
38. What is the purpose of the __manifest__.py file in an Odoo module?. . Easy Answer.	23
39. Differentiate between Many2one and One2many relational fields.. Easy Answer.	23
40. What is the use of __init__.py in an Odoo module?. Easy Answer.	24
41. What is the use of Django signals? Give one example.. Easy Answer.	24
42. Define MEDIA_ROOT and MEDIA_URL in Django.. Easy Answer.	25
43. List different types of Permissions in DRF.. Easy Answer.	25
44. What is a Router in DRF?. Easy Answer.	26
45. Mention two advantages of using JWT Authentication.. Easy Answer.	26
46. Explain the use of computed fields in Odoo with an example.. Easy Answer.	26
47. Write a short note on inheritance in Odoo views.. Easy Answer.	27
48. Explain the use of computed fields in Odoo with an example. Easy Answer.	27
49. Write a short note on inheritance in Odoo views.. Easy Answer.	28
50. Describe the use of QWeb templates in Odoo reporting. Easy Answer.	29
51. Differentiate between Django Models and Migrations.. Easy Answer.	29
52. Write steps to customize Django Admin interface.. Easy Answer.	30
53. Explain CRUD operations in Django using Forms.. Easy Answer.	30
54. Describe Permissions in Django REST Framework with an example.. Easy Answer.	31
55. Explain the use of Nested Serializers in DRF.. Easy Answer.	32
56. Describe different Odoo view types (Tree, Form, Kanban) with suitable use cases.. Easy Answer.	33
57. Explain the use of @api.onchange decorator with an example.. Easy Answer.	34
58. Describe different Odoo view types (Tree, Form, Kanban) with suitable use cases.. Easy Answer.	34
59. Explain the use of @api.onchange decorator with an example.. Easy Answer.	35
60. Explain the difference between model inheritance (_inherit) and class inheritance in Odoo. Easy Answer.	35
61. Explain Django's MVT architecture with a diagram.. Easy Answer.	36
62. Describe the process of creating and applying migrations in Django.. Easy Answer.	37
63. Explain the steps to implement file upload in Django with media management.. Easy Answer.	38
64. Discuss Django's authentication system for login and logout functionality.. Easy Answer.	39
65. Explain Django REST Framework's ViewSets and Routers with an example.. Easy Answer.	40
66. Describe JWT Authentication using djangorestframework-simplejwt with setup steps.. Easy Answer.	41
67. Explain the role of Django templates in the MVT architecture with an example. Easy Answer.	42
68. Describe the process of creating a custom user model in Django.. Easy Answer.	43
69. Explain how to handle file validation in Django forms.. Easy Answer.	43
70. Write token-based authentication vs session-based authentication in Django REST Framework. Easy Answer.	44
71. Write the steps to implement API versioning in Django REST Framework.. Easy Answer.	45
72. Discuss the structure of an Odoo module with examples of essential files. Easy Answer.	46
73. Explain relational fields in Odoo (Many2one, One2many, Many2many) with practical examples.. Easy Answer.	47
74. Explain relational fields in Odoo (Many2one, One2many, Many2many) with practical examples.. Easy Answer.	48
75. Explain the process of creating a custom report in Odoo using QWeb templates.. Easy Answer.	49



1. Which pattern does Django follow?
a) MVVM b) MVC
c) MVP d) MVT

The correct answer is:

d) MVT 

Explanation:

Django follows the **MVT (Model-View-Template)** architectural pattern:

- **Model:** Handles the data and database-related logic.
- **View:** Contains the business logic and interacts with the model to process data.
- **Template:** Manages the presentation layer (HTML) for the user interface.

It is similar to MVC (Model-View-Controller), but in Django, the **Template** acts as the view in MVC, and Django's **View** acts more like the controller.



2. In Django, models are defined in which file by default?
a) views.py b) models.py
c) urls.py d) forms.py

The correct answer is:

b) models.py 

Explanation:

In Django, **models** are Python classes that define the structure of your database tables. By default, these classes are defined in the `models.py` file of each app.

- `views.py` → Handles request/response logic.
- `urls.py` → Maps URLs to views.
- `forms.py` → Defines forms for user input.



3. Which command is used to create migrations in Django?
a) makemodel b) makemigrations
c) migratecreate d) startmigration

The correct answer is:

b) makemigrations 

Explanation:

In Django, the `makemigrations` command is used to create new migrations based on the changes you make to your models. These migrations are Python files that describe the changes to be applied to the database schema.

- **Command:**

```
bash

python manage.py makemigrations
```

After creating migrations, you apply them to the database using:

```
bash

python manage.py migrate
```



4. The Django admin interface can be customized by modifying which file?

- a) admin.py b) models.py
- c) views.py d) settings.py

The correct answer is:

a) admin.py ✓

Explanation:

The **Django admin interface** is customized by modifying the `admin.py` file in your app. You can:

- Register models to appear in the admin.
- Customize how models are displayed (list view, search fields, filters).
- Add custom actions or field formatting.

Example:

```
python

from django.contrib import admin
from .models import Book

@admin.register(Book)
class BookAdmin(admin.ModelAdmin):
    list_display = ('title', 'author', 'published_date')
    search_fields = ('title', 'author')
```



5. Which method is used to save a model instance in Django?

- a) store() b) insert()
- c) save() d) commit()

The correct answer is:

c) save() 

Explanation:

In Django, to save an instance of a model to the database, you use the **save()** method. This method either:

- **Inserts** a new record if the instance is new.
- **Updates** an existing record if the instance already exists.

Example:

```
python

from myapp.models import Book

book = Book(title="Django Basics", author="Harsh")
book.save() # Saves the instance to the database
```



6. In Django, the **render()** function is used to:
- a) Execute queries b) Send email
 - c) Render templates with context d) Create models

The correct answer is:

c) Render templates with context 

Explanation:

In Django, the **render()** function is used in a view to combine a **template** with a **context dictionary** and return an **HttpResponse**. It simplifies the process of rendering HTML pages.

Syntax:

```
python

from django.shortcuts import render

def my_view(request):
    context = {'name': 'Harsh'}
    return render(request, 'my_template.html', context)
```

- **my_template.html** → Template file.
- **context** → Data passed to the template.



7. Which HTTP method is used for updating an object in CRUD operations?
- a) GET b) POST
 - c) PUT/PATCH d) DELETE

The correct answer is:

c) PUT/PATCH ✓

Explanation:

In CRUD operations:

- **Create** → **POST**
- **Read** → **GET**
- **Update** → **PUT or PATCH**
 - **PUT** replaces the entire resource.
 - **PATCH** updates only the specified fields.
- **Delete** → **DELETE**

So, to update an object in RESTful APIs or web applications, **PUT or PATCH** is used.



8. In Django, uploaded files are stored using the settings variable: a) MEDIA_URL b) MEDIA_ROOT c) FILE_PATH d) STATIC_ROOT

The correct answer is:

b) MEDIA_ROOT ✓

Explanation:

In Django:

- **MEDIA_ROOT** → Specifies the **absolute filesystem path** where uploaded files are stored.
- **MEDIA_URL** → URL that handles the media served from **MEDIA_ROOT**.
- **STATIC_ROOT** → Used for static files, not uploaded files.
- **FILE_PATH** → Not a standard Django setting.

Example in settings.py:

```
python

MEDIA_ROOT = BASE_DIR / 'media'
MEDIA_URL = '/media/'
```



9. Which Django module handles user login and logout?
a) django.auth b) django.contrib.auth
c) django.sessions d) django.user

The correct answer is:

b) django.contrib.auth

Explanation:

Django provides a built-in authentication system in the `django.contrib.auth` module, which handles:

- User login and logout
- Password management
- User groups and permissions
- Authentication backends

Example of using login/logout in a view:

```
python

from django.contrib.auth import authenticate, login, logout
from django.shortcuts import redirect

def user_login(request):
    user = authenticate(username='harsh', password='1234')
    if user:
        login(request, user)
        return redirect('home')

def user_logout(request):
    logout(request)
    return redirect('login')
```



10. Which Django framework is used to display temporary success or error messages?

- a) Toasts b) Alerts
c) Messages framework d) Notifications

The correct answer is:

c) Messages framework

Explanation:

Django's **Messages framework** is used to display temporary one-time notifications like success, error, warning, or info messages to users. These messages are stored in the session or cookies and disappear after being displayed.

Example:

```
python

from django.contrib import messages
from django.shortcuts import redirect

def my_view(request):
    messages.success(request, "Profile updated successfully!")
    messages.error(request, "An error occurred.")
    return redirect('home')
```

In the template, you can render them like:

```
html

{% if messages %}
  {% for message in messages %}
    <div class="alert {{ message.tags }}">{{ message }}</div>
  {% endfor %}
{% endif %}
```



11. Django signals are primarily used for:
- a) Inter-process communication b) Triggering actions on certain events
 - c) Debugging d) Logging

The correct answer is:

b) Triggering actions on certain events ✓

Explanation:

Django **signals** allow certain **senders** to notify **receivers** when specific events occur, enabling decoupled event-driven behavior. Common use cases include:

- Automatically creating a profile when a user is created.
- Logging changes when a model instance is saved or deleted.

Example:

```
python

from django.db.models.signals import post_save
from django.contrib.auth.models import User
from django.dispatch import receiver
from .models import Profile

@receiver(post_save, sender=User)
def create_profile(sender, instance, created, **kwargs):
    if created:
        Profile.objects.create(user=instance)
```

Here, the signal triggers **create_profile** whenever a new **User** is created.



12. In DRF, serializers are used for:
- a) Database migrations b) Converting complex data to JSON/XML
 - c) Rendering templates d) Handling sessions

The correct answer is:

b) Converting complex data to JSON/XML ✓

Explanation:

In **Django REST Framework (DRF)**, **serializers** are used to:

- Convert complex Python objects (like Django model instances or querysets) into **JSON, XML, or other content types** for APIs.
- Validate incoming data for creating or updating model instances.

Example:

```
python

from rest_framework import serializers
from .models import Book

class BookSerializer(serializers.ModelSerializer):
    class Meta:
        model = Book
        fields = ['id', 'title', 'author', 'published_date']
```

- **Usage:**

```
python

book = Book.objects.first()
serializer = BookSerializer(book)
print(serializer.data) # Converts Book instance to JSON-like dict
```



13. Which serializer type allows nested relationships in DRF?
- BasicSerializer
 - NestedSerializer
 - ModelSerializer
 - HyperlinkedSerializer

The correct answer is:

c) ModelSerializer

Explanation:

In **Django REST Framework (DRF)**, **ModelSerializer** allows you to easily handle **nested relationships** using related fields such as **ForeignKey**, **ManyToManyField**, or **OneToOneField**. You can also explicitly define nested serializers for more complex structures.

Example:

```
python

from rest_framework import serializers
from .models import Author, Book

class AuthorSerializer(serializers.ModelSerializer):
    class Meta:
        model = Author
        fields = ['id', 'name']

class BookSerializer(serializers.ModelSerializer):
    author = AuthorSerializer() # Nested relationship
    class Meta:
        model = Book
        fields = ['id', 'title', 'author']
```

Here, each **Book** instance will include its related **Author** data in the serialized output.



14. In DRF, ViewSets combine:

- a) Models and Templates b) Views and Routers
- c) CRUD operations into a single class d) Signals and Middleware

The correct answer is:

c) CRUD operations into a single class ✓

Explanation:

In **Django REST Framework (DRF)**, **ViewSets** allow you to combine all the **CRUD operations** (Create, Read, Update, Delete) for a resource into a **single class**, rather than defining separate views for each action.

Example:

```
python

from rest_framework import viewsets
from .models import Book
from .serializers import BookSerializer

class BookViewSet(viewsets.ModelViewSet):
    queryset = Book.objects.all()
    serializer_class = BookSerializer
```

- **ModelViewSet** automatically provides implementations for **list**, **create**, **retrieve**, **update**, and **destroy**.
- Works seamlessly with **routers** to generate URL patterns.



15. DRF Routers are used to:

- a) Manage templates b) Automatically generate URL patterns for ViewSets
- c) Handle database queries d) Manage signals

The correct answer is:

b) Automatically generate URL patterns for ViewSets ✓

Explanation:

In **Django REST Framework (DRF)**, **routers** simplify the process of URL routing for **ViewSets** by automatically generating the necessary URL patterns for all the standard actions (**list**, **create**, **retrieve**, **update**, **destroy**).

Example:

```
python

from rest_framework import routers
from .views import BookViewSet

router = routers.DefaultRouter()
```

```
router.register(r'books', BookViewSet)

urlpatterns = router.urls
```

This will create URLs like:

- `/books/` → list and create
- `/books/<id>/` → retrieve, update, destroy

No need to manually define URLs for each action.



16. Which DRF class controls access to API endpoints?
 a) Views b) Routers
 c) Permissions d) Middleware

The correct answer is:

c) Permissions ✓

Explanation:

In **Django REST Framework (DRF)**, **permissions** control **who can access API endpoints**. They determine whether a request should be granted or denied based on the user, request method, or custom rules.

Common built-in permissions:

- **AllowAny** → Anyone can access.
- **IsAuthenticated** → Only logged-in users can access.
- **IsAdminUser** → Only admin users can access.
- **IsAuthenticatedOrReadOnly** → Read access for everyone, write access for authenticated users.

Example:

```
python

from rest_framework.permissions import IsAuthenticated
from rest_framework import viewsets
from .models import Book
from .serializers import BookSerializer

class BookViewSet(viewsets.ModelViewSet):
    queryset = Book.objects.all()
    serializer_class = BookSerializer
    permission_classes = [IsAuthenticated] # Restrict access
```



17. JWT stands for:
 a) Java Web Tokens b) JSON Web Tokens
 c) JavaScript Web Transport d) JSON Writable Type

The correct answer is:

b) JSON Web Tokens

Explanation:

JWT (JSON Web Token) is a compact, URL-safe token format used for **securely transmitting information between parties** as a JSON object. It is widely used for **authentication and authorization** in web applications.

- Consists of three parts: **Header, Payload, Signature**.
- Commonly used in **Django REST Framework** for stateless authentication.

Example structure:

```
nginx
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
.eyJ1c2VyX2lkIjozLCJleHAiOjE2Nzk2Mjc5ODV9
.sflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c
```



18. Which DRF package is used for JWT Authentication?

- a) django-rest-framework-jwt b) django-jwt-auth
c) rest_framework_simplejwt d) jwt-drf

The correct answer is:

c) rest_framework_simplejwt

Explanation:

In **Django REST Framework (DRF)**, the **rest_framework_simplejwt** package is the recommended and actively maintained package for **JWT authentication**. It provides:

- Token generation (**access** and **refresh** tokens)
- Token verification
- Token refresh and rotation
- Customizable token lifetimes

Example setup in settings.py:

```
python

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ),
}
```

Example URL configuration:

```
python
```

```
from rest_framework_simplejwt.views import TokenObtainPairView, TokenRefreshView

urlpatterns = [
    path('api/token/', TokenObtainPairView.as_view(), name='token_obtain_pair'),
    path('api/token/refresh/', TokenRefreshView.as_view(), name='token_refresh'),
]
```



19. The default HTTP method allowed in DRF ViewSets is:

- a) GET b) POST
- c) GET, POST, PUT, DELETE d) Only GET and POST

The correct answer is:

c) GET, POST, PUT, DELETE ✓

Explanation:

In **Django REST Framework (DRF)**, **ModelViewSet** (the most common type of ViewSet) provides **all standard CRUD operations** by default:

- **GET** → List or retrieve objects
- **POST** → Create objects
- **PUT/PATCH** → Update objects
- **DELETE** → Delete objects

So, by default, a ViewSet supports **all these HTTP methods**, and you can restrict them using **http_method_names** or **permissions** if needed.



20. In DRF, the **is_authenticated** attribute belongs to:

- a) request.user b) request.session
- c) serializer.data d) view.router

The correct answer is:

a) request.user ✓

Explanation:

In **Django REST Framework (DRF)**, the **is_authenticated** attribute is part of the **User object** (**request.user**). It indicates whether the current user is logged in or anonymous.

Example:

```
python
```

```
from rest_framework.views import APIView
from rest_framework.response import Response

class MyView(APIView):
```

```
def get(self, request):
    if request.user.is_authenticated:
        return Response({"message": "Hello, authenticated user!"})
    else:
        return Response({"message": "Hello, guest!"})
```

- `request.session` → Manages session data.
- `serializer.data` → Serialized output of data.
- `view.router` → Not a standard DRF attribute.



21. In Django, which signal is triggered after a model instance is saved?

- a) `pre_save` b) `post_save`
- c) `save_signal` d) `after_save`

The correct answer is:

b) `post_save` ✓

Explanation:

In Django, the `post_save` signal is triggered **after a model instance is saved** to the database. It can be used to perform actions such as creating related objects, sending notifications, or logging.

Example:

```
python

from django.db.models.signals import post_save
from django.dispatch import receiver
from .models import Book

@receiver(post_save, sender=Book)
def after_book_saved(sender, instance, created, **kwargs):
    if created:
        print(f"New book created: {instance.title}")
    else:
        print(f"Book updated: {instance.title}")
```

- `pre_save` → Triggered **before** saving the instance.
- `save_signal` and `after_save` → Not standard Django signals.



22. In DRF, which permission class grants read-only access to unauthenticated users but write access to authenticated users?

- a) `IsAuthenticated` b) `IsAdminUser`
- c) `AllowAny` d) `IsAuthenticatedOrReadOnly`

The correct answer is:

d) `IsAuthenticatedOrReadOnly` ✓

Explanation:

In **Django REST Framework (DRF)**:

- **IsAuthenticatedOrReadOnly** allows:
 - **Read operations** (GET, HEAD, OPTIONS) for **all users** (authenticated or not).
 - **Write operations** (POST, PUT, PATCH, DELETE) only for **authenticated users**.

Example:

```
python

from rest_framework.permissions import IsAuthenticatedOrReadOnly
from rest_framework import viewsets
from .models import Book
from .serializers import BookSerializer

class BookViewSet(viewsets.ModelViewSet):
    queryset = Book.objects.all()
    serializer_class = BookSerializer
    permission_classes = [IsAuthenticatedOrReadOnly]
```

This is useful for APIs where anyone can view data but only logged-in users can modify it.



23. Which file in an Odoo module defines the basic metadata (name, version, dependencies)?

- a) `__init__.py` b) `__manifest__.py`
- c) `models.py` d) `views.xml`

The correct answer is:

b) `__manifest__.py` ✓

Explanation:

In an **Odoo module**, the `__manifest__.py` file (formerly `__openerp__.py`) contains the module's **metadata**, including:

- **name** → Module name
- **version** → Module version
- **depends** → List of other modules this module depends on
- **author, description, data**, etc.

Example:

```
python

{
    'name': 'Library Management',
    'version': '1.0',
    'depends': ['base'],
    'author': 'Harsh',
    'category': 'Tools',
    'data': ['views/library_book_views.xml'],
}
```

- `__init__.py` → Initializes Python packages.
- `models.py` → Defines model classes.
- `views.xml` → Defines the UI views.



24. Which relational field in Odoo defines a many-to-one relationship?
 a) Many2many b) Many2one
 c) One2many d) Char

The correct answer is:

b) Many2one ✓

Explanation:

In **Odoo**, the **Many2one** field defines a **many-to-one relationship**, meaning:

- Many records of the current model can be linked to **one record of another model**.

Example:

```
python

from odoo import models, fields

class Book(models.Model):
    _name = 'library.book'
    _description = 'Book'

    name = fields.Char(string='Title')
    author_id = fields.Many2one('res.partner', string='Author') # Many books → one author
```

- **One2many** → One record links to many records.
- **Many2many** → Many records link to many records.
- **Char** → Simple text field, not relational.



25. Which Odoo view type is used to display data in a tabular format?
 a) Kanban b) Form
 c) Tree d) Calendar

The correct answer is:

c) Tree ✓

Explanation:

In **Odoo**, the **Tree view** (also called **List view**) is used to display records in a **tabular format**, showing multiple records at once with columns for different fields.

Other view types:

- **Form view** → Displays details of a single record.

- **Kanban view** → Card-based layout for quick overview.
- **Calendar view** → Displays records with date fields on a calendar.

Example (tree view in XML):

```
xml

<record id="view_library_book_tree" model="ir.ui.view">
  <field name="name">library.book.tree</field>
  <field name="model">library.book</field>
  <field name="arch" type="xml">
    <tree>
      <field name="name"/>
      <field name="author_id"/>
      <field name="published_date"/>
    </tree>
  </field>
</record>
```



26. Which decorator is used in Odoo to compute field values dynamically?
- a) @onchange b) @compute
c) @depends d) @model

The correct answer is:

b) @compute ✓

Explanation:

In **Odoo**, the **@compute** decorator is used to define a method that **dynamically computes the value of a field** based on other fields.

Example:

```
python

from odoo import models, fields, api

class SaleOrder(models.Model):
    _name = 'sale.order'

    amount_total = fields.Float(string='Total Amount', compute='_compute_total')

    @api.depends('order_line.price_total')
    def _compute_total(self):
        for order in self:
            order.amount_total = sum(line.price_total for line in order.order_line)
```

- **@depends** → Used **inside** compute methods to declare dependencies.
- **@onchange** → Triggered when a field value changes in the form view.
- **@model** → Not a standard Odoo decorator.



27. In QWeb templates, which symbol is used for dynamic value rendering?

- a) {% %}
- b) [[]]
- c) t-esc
- d) <? ?>

The correct answer is:

c) t-esc 

Explanation:

In **Odoo QWeb templates**, **t-esc** is used to **render dynamic values safely** in the template. It escapes HTML characters to prevent XSS attacks.

Example:

```
xml

<t t-name="my_module.template_example">
  <div>
    <span t-esc="object.name"/>
  </div>
</t>
```

- **t-esc** → Escapes and displays the value of `object.name`.
- **t-raw** → Displays the value without escaping HTML.
- **{% %}** → Used in Django templates, not QWeb.
- **[[]]** → Used in some JS frameworks, not QWeb.



28. Which relational field is used when a record can have multiple associated records in both directions?

- a) Many2one
- b) One2many
- c) Many2many
- d) Char

The correct answer is:

c) Many2many 

Explanation:

In **Odoo**, the **Many2many** field is used when a record can be associated with **multiple records in both directions**. This creates a **bidirectional relationship** between two models.

Example:

```
python

from odoo import models, fields

class Student(models.Model):
    _name = 'school.student'

    name = fields.Char(string='Name')
    course_ids = fields.Many2many('school.course', string='Courses') # Student can have many courses
```

```
class Course(models.Model):
    _name = 'school.course'

    name = fields.Char(string='Course Name')
    student_ids = fields.Many2many('school.student', string='Students') # Course can have many
students
```

- **Many2one** → Many records point to **one** record.
- **One2many** → One record points to **many** records.
- **Char** → Simple text field, not relational.



29. Which of the following is true about @api.onchange in Odoo?

- It is triggered when the record is saved
- It is triggered immediately after a field value changes
- It computes values automatically on all records
- It replaces @compute decorator

The correct answer is:

b) It is triggered immediately after a field value changes ✓

Explanation:

In **Odoo**, the **@api.onchange** decorator is used to **trigger a method whenever a specified field value changes in the form view**. It allows you to dynamically update other fields **before saving the record**.

Example:

```
python

from odoo import models, fields, api

class SaleOrder(models.Model):
    _name = 'sale.order'

    price_unit = fields.Float(string='Unit Price')
    quantity = fields.Float(string='Quantity')
    total_price = fields.Float(string='Total Price')

    @api.onchange('price_unit', 'quantity')
    def _onchange_total_price(self):
        for record in self:
            record.total_price = record.price_unit * record.quantity
```

- **a)** → Incorrect, **@api.onchange** is not triggered on save.
- **c)** → Incorrect, it only affects the current form record.
- **d)** → Incorrect, it does not replace **@compute**.



30. In Odoo, which file is mandatory to load views?

- controllers.py
- __init__.py
- XML file inside views/ folder
- README file

The correct answer is:

c) XML file inside views/ folder

Explanation:

In **Odoo**, **views are defined in XML files**, usually placed inside the **views/** folder of a module. These files are **mandatory to load the UI views** (form, tree, kanban, etc.) and must be referenced in the module's **__manifest__.py** under the **data** key.

Example in __manifest__.py:

```
python

'data': [
    'views/library_book_views.xml',
]
```

- **controllers.py** → Defines backend routes, not mandatory for views.
- **__init__.py** → Initializes Python packages.
- **README** → Documentation only, not used to load views.



31. What does MVC stand for in Django context?. Easy Answer

In the **Django context**, MVC stands for:

M → **Model**

- Handles the **data and database logic**.

V → **View**

- Contains the **business logic** and processes user requests.

C → **Controller**

- Manages the **flow of data between Model and View**.
- In Django, the **View acts like the Controller**, and the **Template acts like the View** in classic MVC.

So Django's architecture is often called **MVT (Model-View-Template)**, which is similar to MVC.



32. Which command applies migrations in Django?. Easy Answer

The command to apply migrations in Django is:

```
bash

python manage.py migrate
\,\, 
```

```
**Explanation:**
- This command **applies all pending migrations** to the database.
- It creates or updates database tables according to the changes defined in your models.

- **`makemigrations`** → Creates migration files.
- **`migrate`** → Applies them to the database.
```



33. What is the role of `ModelSerializer` in DRF?. Easy Answer

In **Django REST Framework (DRF)**, the `ModelSerializer` is used to:

- **Automatically convert Django model instances into JSON** (or other formats) for APIs.
- **Validate incoming data** and create/update model instances.
- **Simplify serialization** by automatically generating fields based on the model.

Example:

```
python

from rest_framework import serializers
from .models import Book

class BookSerializer(serializers.ModelSerializer):
    class Meta:
        model = Book
        fields = ['id', 'title', 'author']
```

It saves time compared to writing a serializer manually.



34. Name two built-in authentication classes in DRF.. Easy Answer

Two built-in authentication classes in **Django REST Framework (DRF)** are:

1. **SessionAuthentication** ✓
 - Uses Django's **session framework** for authentication (like logging in via the website).
2. **BasicAuthentication** ✓
 - Uses **HTTP basic authentication** with username and password sent in the request header.

Other examples include **TokenAuthentication** and **JWTAuthentication** (with packages).



35. What is the default port for Django's development server?. Easy Answer

The default port for Django's development server is:

8000 ✓

Usage:

```
bash

python manage.py runserver
```

- By default, it runs on <http://127.0.0.1:8000/> ↗
- You can change the port by specifying it:

```
bash

python manage.py runserver 8080
```



36. What is the purpose of the `__manifest__.py` file in an Odoo module?. Easy Answer

The `__manifest__.py` file in an Odoo module is used to define the module's **metadata**. ✓

Purpose:

- Provides information about the module such as:
 - **name** → Module name
 - **version** → Module version
 - **depends** → List of other modules it depends on
 - **author, category, description**
 - **data** → Files to load (views, security, etc.)

Example:

```
python

{
    'name': 'Library Management',
    'version': '1.0',
    'depends': ['base'],
    'author': 'Harsh',
    'data': ['views/library_book_views.xml'],
}
```

It is **mandatory** for Odoo to recognize and install the module.



37. Differentiate between Many2one and One2many relational fields.. Easy Answer

Difference between Many2one and One2many in Odoo: ✓

Field Type	Relationship Direction	Example Use Case
Many2one	Many records → One record	Many books belong to one author
One2many	One record → Many records	One author has many books

Explanation:

- **Many2one** creates a **single link** from the current model to another model.
- **One2many** creates a **reverse link**, showing all related records of another model.

Example:

```
python

class Book(models.Model):
    _name = 'library.book'
    author_id = fields.Many2one('res.partner', string='Author') # Many books → one author

class Author(models.Model):
    _name = 'res.partner'
    book_ids = fields.One2many('library.book', 'author_id', string='Books') # One author → many books
```



38. What is the purpose of the `__manifest__.py` file in an Odoo module?. . Easy Answer

The `__manifest__.py` file in an Odoo module is used to define the module's **metadata**. 

Purpose:

- Specifies the **name, version, and description** of the module.
- Lists **dependencies** on other modules.
- Defines **data files** to load (like views, security rules).
- Provides **author and category information**.

Example:

```
python

{
    'name': 'Library Management',
    'version': '1.0',
    'depends': ['base'],
    'author': 'Harsh',
    'data': ['views/library_book_views.xml'],
}
```

It is **mandatory** for Odoo to recognize and install the module.



39. Differentiate between Many2one and One2many relational fields.. Easy Answer

Difference between Many2one and One2many in Odoo:

Field Type	Relationship Direction	Example Use Case
Many2one	Many records → One record	Many books belong to one author
One2many	One record → Many records	One author has many books

Explanation:

- **Many2one** links a record to **a single record** in another model.
- **One2many** links a record to **multiple records** in another model (reverse of Many2one).

Example:

```
python

class Book(models.Model):
    _name = 'library.book'
    author_id = fields.Many2one('res.partner', string='Author') # Many books → one author

class Author(models.Model):
    _name = 'res.partner'
    book_ids = fields.One2many('library.book', 'author_id', string='Books') # One author → many books
```



40. What is the use of `__init__.py` in an Odoo module?. Easy Answer

The `__init__.py` file in an Odoo module is used to **initialize Python packages**. 

Purpose:

- Tells Python that the folder is a **package**.
- Imports **models, controllers, and other Python files** so Odoo can use them.

Example:

```
python

# __init__.py
from . import models
from . import controllers
```

- Without `__init__.py`, Python and Odoo won't recognize the module's Python files.



41. What is the use of Django signals? Give one example.. Easy Answer

Use of Django signals:

Django signals are used to **trigger actions automatically when certain events occur** in the application, allowing decoupled communication between components.

Example: Trigger an action after a user is created.

```
python

from django.db.models.signals import post_save
from django.contrib.auth.models import User
from django.dispatch import receiver

@receiver(post_save, sender=User)
def create_profile(sender, instance, created, **kwargs):
    if created:
        print(f"Profile created for user: {instance.username}")
```

- Here, the signal **runs automatically after a new User is saved**.



42. Define MEDIA_ROOT and MEDIA_URL in Django.. Easy Answer

MEDIA_ROOT and MEDIA_URL in Django:

- **MEDIA_ROOT** → The **absolute filesystem path** where uploaded media files (images, documents) are stored.
- **MEDIA_URL** → The **URL path** used to access these uploaded files in the browser.

Example in settings.py:

```
python

MEDIA_ROOT = BASE_DIR / 'media'
MEDIA_URL = '/media/'
```

- Uploaded files are saved in **MEDIA_ROOT**.
- They can be accessed via URLs starting with **MEDIA_URL**.



43. List different types of Permissions in DRF.. Easy Answer

Different types of permissions in Django REST Framework (DRF):

1. **AllowAny** → Anyone can access the API, authenticated or not.
 2. **IsAuthenticated** → Only logged-in users can access.
 3. **IsAdminUser** → Only admin users can access.
 4. **IsAuthenticatedOrReadOnly** → Read access for everyone, write access for authenticated users.
- You can also create **custom permission classes** for more specific rules.



44. What is a Router in DRF?. Easy Answer

Router in Django REST Framework (DRF):

A **Router** is used to **automatically generate URL patterns** for **ViewSets**, so you don't have to define each URL manually.

Example:

```
python

from rest_framework import routers
from .views import BookViewSet

router = routers.DefaultRouter()
router.register(r'books', BookViewSet)

urlpatterns = router.urls
```

- Automatically creates URLs for **list, create, retrieve, update, and delete** actions.
- Simplifies API routing.



45. Mention two advantages of using JWT Authentication.. Easy Answer

Two advantages of JWT (JSON Web Token) Authentication:

1. **Stateless Authentication** – No need to store session data on the server; all information is in the token.
 2. **Cross-platform / Scalable** – Works across different domains and services, making it ideal for APIs and microservices.
- Additional benefits: secure, compact, and can include custom user data in the token payload.



46. Explain the use of computed fields in Odoo with an example.. Easy Answer

Computed fields in Odoo:

Use:

- Computed fields are **dynamically calculated** based on other fields.
- They are **not stored** in the database unless **store=True** is specified.

Example: Calculate the total price of a sale order.

```
python

from odoo import models, fields, api
```

```
class SaleOrder(models.Model):
    _name = 'sale.order'

    price_unit = fields.Float(string='Unit Price')
    quantity = fields.Float(string='Quantity')
    total_price = fields.Float(string='Total Price', compute='_compute_total')

    @api.depends('price_unit', 'quantity')
    def _compute_total(self):
        for order in self:
            order.total_price = order.price_unit * order.quantity
```

- Here, **total_price** is automatically calculated whenever **price_unit** or **quantity** changes.



47. Write a short note on inheritance in Odoo views.. Easy Answer

Inheritance in Odoo Views:

Use:

- Allows you to **modify or extend existing views** without rewriting them completely.
- Useful for customizing modules safely.

Types:

1. **<xpath> inheritance** – Insert, replace, or remove elements using XPath.
2. **position attribute** – Specify where to apply changes (**inside, before, after, replace**).

Example: Add a field to an existing form view.

```
xml

<record id="view_order_form_inherit" model="ir.ui.view">
    <field name="name">sale.order.form.inherit</field>
    <field name="model">sale.order</field>
    <field name="inherit_id" ref="sale.view_order_form"/>
    <field name="arch" type="xml">
        <xpath expr="//field[@name='partner_id']" position="after">
            <field name="delivery_date"/>
        </xpath>
    </field>
</record>
```

- This adds **delivery_date** after **partner_id** in the existing Sale Order form.



48. Explain the use of computed fields in Odoo with an example. Easy Answer

Computed Fields in Odoo:

Use:

- Computed fields are **dynamically calculated** based on other fields.

- They are useful for values that depend on other data and don't need manual entry.
- Can be **stored** in the database using **store=True** if needed for searching or grouping.

Example: Calculate total price of an order:

```
python

from odoo import models, fields, api

class SaleOrder(models.Model):
    _name = 'sale.order'

    price_unit = fields.Float(string='Unit Price')
    quantity = fields.Float(string='Quantity')
    total_price = fields.Float(string='Total Price', compute='_compute_total')

    @api.depends('price_unit', 'quantity')
    def _compute_total(self):
        for order in self:
            order.total_price = order.price_unit * order.quantity
```

- **total_price** is automatically computed whenever **price_unit** or **quantity** changes.



49. Write a short note on inheritance in Odoo views.. Easy Answer

Inheritance in Odoo Views:

Use:

- Inheritance allows you to **modify or extend existing views** without rewriting them completely.
- It helps in **customizing modules safely**.

How it works:

1. **inherit_id** → References the existing view to extend.
2. **<xpath> expressions** → Locate elements in the original view to modify.
3. **position attribute** → Specifies where to insert, replace, or remove elements (**inside**, **before**, **after**, **replace**).

Example: Add a field to an existing form view:

```
xml

<record id="view_order_form_inherit" model="ir.ui.view">
    <field name="name">sale.order.form.inherit</field>
    <field name="model">sale.order</field>
    <field name="inherit_id" ref="sale.view_order_form"/>
    <field name="arch" type="xml">
        <xpath expr="//field[@name='partner_id']" position="after">
            <field name="delivery_date"/>
        </xpath>
    </field>
</record>
```

- This adds the **delivery_date** field after **partner_id** in the Sale Order form.



50. Describe the use of QWeb templates in Odoo reporting. Easy Answer

QWeb Templates in Odoo Reporting:

Use:

- QWeb is Odoo's **templating engine** used to **design HTML reports**.
- Allows you to **dynamically display data** from models in a structured format.
- Supports loops, conditions, and expressions to render complex reports.

Example: Generate a simple report of books:

```
xml

<t t-name="library.report_books">
  <t t-foreach="docs" t-as="book">
    <p><t t-esc="book.name"/> by <t t-esc="book.author_id.name"/></p>
  </t>
</t>
```

- **t-foreach** → Loop through records.
- **t-esc** → Display dynamic values safely.

QWeb templates are widely used for **PDF, HTML, and printed reports** in Odoo.



51. Differentiate between Django Models and Migrations.. Easy Answer

Difference between Django Models and Migrations:

Aspect	Models	Migrations
Definition	Python classes that define database structure (tables and fields).	Files that record changes to models and apply them to the database.
Purpose	Describe how data is stored and related.	Keep the database schema in sync with models.
Creation	Created manually in models.py .	Created using python manage.py makemigrations .
Application	Not applied directly to the database.	Applied using python manage.py migrate to update the database.

- **Models** → Define the **structure**.
- **Migrations** → Make the **actual changes** in the database.



52. Write steps to customize Django Admin interface.. Easy Answer

Steps to Customize Django Admin Interface:

1. Import admin and your model

```
python

from django.contrib import admin
from .models import Book
```

2. Create a custom admin class

```
python

class BookAdmin(admin.ModelAdmin):
    list_display = ('title', 'author', 'published_date') # Columns to show
    search_fields = ('title', 'author') # Searchable fields
    list_filter = ('published_date',) # Filters
```

3. Register the model with the custom admin class

```
python

admin.site.register(Book, BookAdmin)
```

4. Optional Customizations:

- Add **actions**
- Customize **form layout**
- Use **fieldsets** to group fields

This allows you to **control how models appear and behave** in Django's admin panel.



53. Explain CRUD operations in Django using Forms.. Easy Answer

CRUD Operations in Django Using Forms:

CRUD stands for **Create, Read, Update, Delete**. In Django, forms are commonly used to perform these operations.

1. Create (Add a record)

```
python

# views.py
from django.shortcuts import render, redirect
from .forms import BookForm

def add_book(request):
```

```
if request.method == 'POST':
    form = BookForm(request.POST)
    if form.is_valid():
        form.save() # Create new record
        return redirect('book_list')
else:
    form = BookForm()
return render(request, 'add_book.html', {'form': form})
```

2. Read (View records)

python

```
def book_list(request):
    books = Book.objects.all() # Retrieve all records
    return render(request, 'book_list.html', {'books': books})
```

3. Update (Edit a record)

python

```
def edit_book(request, pk):
    book = Book.objects.get(id=pk)
    form = BookForm(request.POST or None, instance=book)
    if form.is_valid():
        form.save() # Update record
        return redirect('book_list')
    return render(request, 'edit_book.html', {'form': form})
```

4. Delete (Remove a record)

python

```
def delete_book(request, pk):
    book = Book.objects.get(id=pk)
    book.delete() # Delete record
    return redirect('book_list')
```

- **Forms** handle validation and data input.
- **Models** interact with the database.
- Templates render the forms and display data.



54. Describe Permissions in Django REST Framework with an example.. Easy Answer

Permissions in Django REST Framework (DRF):

Use:

- Permissions control **who can access API endpoints**.
- They help secure APIs by restricting access based on user roles, authentication, or custom rules.

Common Built-in Permissions:

1. **AllowAny** → Anyone can access the API.
2. **IsAuthenticated** → Only logged-in users can access.
3. **IsAdminUser** → Only admin users can access.
4. **IsAuthenticatedOrReadOnly** → Read access for all, write access for authenticated users.

Example: Restrict access to authenticated users only

python

```
from rest_framework.permissions import IsAuthenticated
from rest_framework import viewsets
from .models import Book
from .serializers import BookSerializer

class BookViewSet(viewsets.ModelViewSet):
    queryset = Book.objects.all()
    serializer_class = BookSerializer
    permission_classes = [IsAuthenticated] # Only authenticated users can access
```

- **permission_classes** is set at the **ViewSet or APIView level**.
- Custom permissions can also be created by subclassing **BasePermission**.



55. Explain the use of Nested Serializers in DRF.. Easy Answer

Nested Serializers in Django REST Framework (DRF):

Use:

- Nested serializers allow you to **include related model data** within a parent serializer.
- Useful for displaying **complex relationships** like **ForeignKey** or **ManyToMany** in JSON responses.

Example:

python

```
from rest_framework import serializers
from .models import Author, Book

class AuthorSerializer(serializers.ModelSerializer):
    class Meta:
        model = Author
        fields = ['id', 'name']

class BookSerializer(serializers.ModelSerializer):
    author = AuthorSerializer() # Nested serializer
    class Meta:
        model = Book
        fields = ['id', 'title', 'author']
```

- Each **Book** in the API response will include its related **Author** data.
- Helps structure **hierarchical data** in API responses cleanly.



56. Describe different Odoo view types (Tree, Form, Kanban) with suitable use cases.. Easy Answer

Different Odoo View Types:

1. Tree View (List View)

- **Use:** Display multiple records in a **tabular format**.
- **Example:** Listing all books in a library with columns for title, author, and published date.

```
xml

<tree>
  <field name="name"/>
  <field name="author_id"/>
  <field name="published_date"/>
</tree>
```

2. Form View

- **Use:** Display **detailed information** of a single record.
- **Example:** Viewing/editing the details of a single book, including description and ISBN.

```
xml

<form>
  <field name="name"/>
  <field name="author_id"/>
  <field name="description"/>
</form>
```

3. Kanban View

- **Use:** Display records as **cards**, useful for workflow or quick overview.
- **Example:** Managing tasks or sales orders in stages.

```
xml

<kanban>
  <field name="name"/>
  <field name="stage_id"/>
</kanban>
```

Summary:

- **Tree:** Quick overview of many records.
- **Form:** Detailed single record view.
- **Kanban:** Visual management of records in a card layout.



57. Explain the use of @api.onchange decorator with an example.. Easy Answer

@api.onchange in Odoo: 

Use:

- Trigger a method **automatically when a field value changes** in the form view.
- Useful for **updating other fields dynamically** before saving the record.

Example: Calculate total price when quantity or unit price changes

```
python

from odoo import models, fields, api

class SaleOrder(models.Model):
    _name = 'sale.order'

    price_unit = fields.Float(string='Unit Price')
    quantity = fields.Float(string='Quantity')
    total_price = fields.Float(string='Total Price')

    @api.onchange('price_unit', 'quantity')
    def _onchange_total_price(self):
        for order in self:
            order.total_price = order.price_unit * order.quantity
```

- Here, **total_price updates automatically** whenever **price_unit** or **quantity** changes in the form view.
- Unlike @compute, this is **triggered only in the form view**, not automatically on all records.



58. Describe different Odoo view types (Tree, Form, Kanban) with suitable use cases.. Easy Answer

Different Odoo View Types: 

1. Tree View (List View)

- **Purpose:** Show multiple records in a **tabular format**.
- **Use Case:** Listing all books in a library with columns like title, author, and published date.

2. Form View

- **Purpose:** Display **detailed information** of a single record.
- **Use Case:** Viewing or editing the details of a single book, including description and ISBN.

3. Kanban View

- **Purpose:** Show records as **cards**, useful for workflow or quick overview.
- **Use Case:** Managing tasks or sales orders in stages, moving cards across columns.

Summary:

- **Tree:** Quick overview of many records.
- **Form:** Detailed single record view.
- **Kanban:** Visual management of records in card layout.



59. Explain the use of @api.onchange decorator with an example.. Easy Answer

@api.onchange in Odoo: 

Use:

- Automatically triggers a method **when a specified field value changes** in the form view.
- Useful for **dynamically updating other fields** based on user input before saving.

Example: Calculate total price when quantity or unit price changes

python

```
from odoo import models, fields, api

class SaleOrder(models.Model):
    _name = 'sale.order'

    price_unit = fields.Float(string='Unit Price')
    quantity = fields.Float(string='Quantity')
    total_price = fields.Float(string='Total Price')

    @api.onchange('price_unit', 'quantity')
    def _onchange_total_price(self):
        for order in self:
            order.total_price = order.price_unit * order.quantity
```

- Here, **total_price updates automatically** whenever **price_unit** or **quantity** changes in the form view.
- Unlike @compute, this is **triggered only in the form view**, not on all records in the database.



60. Explain the difference between model inheritance (_inherit) and class inheritance in Odoo. Easy Answer

Difference between Model Inheritance (_inherit) and Class Inheritance in Odoo: 

Feature	Class Inheritance	Model Inheritance (_inherit)
Purpose	Reuse Python code and methods	Extend or modify an existing Odoo model
How it works	Standard Python inheritance (class Child(Parent))	Use _inherit = 'existing.model' in a model
Effect	Adds or overrides methods and attributes in Python	Adds new fields, methods, or overrides database model features

Feature	Class Inheritance	Model Inheritance (<code>_inherit</code>)
Example		

python

```
class MyModel(models.Model, BaseModel):
    pass
'''python
class SaleOrder(models.Model):
    _inherit = 'sale.order'
    new_field = fields.Char()
'''

**Summary:**
- **Class inheritance** → Python-level code reuse.
- **Model inheritance (_inherit)** → Extend existing Odoo models and their database structure.
```



61. Explain Django's MVT architecture with a diagram.. Easy Answer

Django's MVT Architecture:

MVT stands for **Model-View-Template** and is similar to MVC.

1. Model

- Represents **database structure** and data logic.
- Example: **Book** model with fields like **title**, **author**.

2. View

- Handles **business logic**, processes requests, and returns responses.
- Example: A function or class-based view that retrieves books and sends them to a template.

3. Template

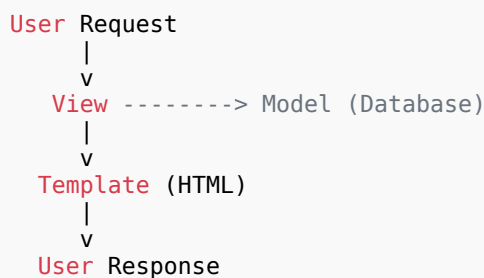
- Handles **presentation**, i.e., what the user sees (HTML).
- Example: `book_list.html` displaying a list of books.

Flow:

1. User sends a request →
2. **View** processes it →
3. Interacts with **Model** →
4. Sends data to **Template** →
5. Rendered HTML returned to user.

Diagram:

```
pgsql
```



- Django's **View acts like Controller** in MVC.
- Template handles **UI**, Model handles **data**.



62. Describe the process of creating and applying migrations in Django.. Easy Answer

Creating and Applying Migrations in Django:

1. Creating Migrations

- Migrations are **Python files that record changes** to your models.
- Command:

```
bash
```

```
python manage.py makemigrations
```

- Django detects changes in `models.py` and creates migration files in the `migrations/` folder.

2. Applying Migrations

- Apply the migrations to the database to **update schema**.
- Command:

```
bash
```

```
python manage.py migrate
```

- Creates or alters database tables based on migration files.

Summary:

- `makemigrations` → Generate migration files.
- `migrate` → Apply changes to the database.

This ensures **models and database schema stay in sync**.



63. Explain the steps to implement file upload in Django with media management.. Easy Answer

Steps to Implement File Upload in Django with Media Management:

1. Set up media settings in settings.py

```
python

MEDIA_URL = '/media/' # URL to access uploaded files
MEDIA_ROOT = BASE_DIR / 'media' # Directory to store uploaded files
```

2. Create a model with FileField or ImageField

```
python

from django.db import models

class Document(models.Model):
    title = models.CharField(max_length=100)
    file = models.FileField(upload_to='documents/')
```

3. Create a form

```
python

from django import forms
from .models import Document

class DocumentForm(forms.ModelForm):
    class Meta:
        model = Document
        fields = ['title', 'file']
```

4. Handle file upload in a view

```
python

from django.shortcuts import render, redirect
from .forms import DocumentForm

def upload_document(request):
    if request.method == 'POST':
        form = DocumentForm(request.POST, request.FILES)
        if form.is_valid():
            form.save()
            return redirect('document_list')
    else:
        form = DocumentForm()
    return render(request, 'upload.html', {'form': form})
```

5. Update URLs to serve media files during development

```
python

from django.conf import settings
from django.conf.urls.static import static
```

```
urlpatterns = [
    # ... your urls
] + static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

6. Create template (upload.html)

```
html

<form method="post" enctype="multipart/form-data">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Upload</button>
</form>
```

- `upload_to` in `FileField` defines subdirectory.
- `enctype="multipart/form-data"` is required in the form.

This setup allows users to **upload and access files** via Django's media management.



64. Discuss Django's authentication system for login and logout functionality.. Easy Answer

Django's Authentication System for Login and Logout:

Purpose:

- Handles **user authentication**, including login, logout, and session management.
- Provides ready-made **User model** and views for authentication.

1. Login Functionality

- Use Django's `authenticate()` and `login()` functions.

```
python

from django.contrib.auth import authenticate, login
from django.shortcuts import render, redirect

def user_login(request):
    if request.method == 'POST':
        username = request.POST['username']
        password = request.POST['password']
        user = authenticate(request, username=username, password=password)
        if user is not None:
            login(request, user) # Log the user in
            return redirect('home')
        return render(request, 'login.html')
```

2. Logout Functionality

- Use Django's `logout()` function to end the session.

```
python

from django.contrib.auth import logout
from django.shortcuts import redirect
```

```
def user_logout(request):
    logout(request) # Log the user out
    return redirect('login')
```

3. Templates

- Provide HTML forms for login with fields **username** and **password**.

Summary:

- **authenticate()** → Verifies credentials.
- **login()** → Creates a session for the user.
- **logout()** → Ends the session.

This system ensures secure **user login and logout** with session management.



65. Explain Django REST Framework's ViewSets and Routers with an example.. Easy Answer

Django REST Framework (DRF) ViewSets and Routers:

1. ViewSets

- Combine **CRUD operations** (Create, Read, Update, Delete) into a **single class**.
- Simplifies views by avoiding separate methods for each HTTP action.

Example:

```
python

from rest_framework import viewsets
from .models import Book
from .serializers import BookSerializer

class BookViewSet(viewsets.ModelViewSet):
    queryset = Book.objects.all()
    serializer_class = BookSerializer
```

2. Routers

- Automatically generate **URL patterns** for ViewSets.
- No need to manually define URLs for each action.

Example:

```
python

from rest_framework import routers
from .views import BookViewSet

router = routers.DefaultRouter()
router.register(r'books', BookViewSet)
```

```
urlpatterns = router.urls
```

Summary:

- **ViewSet:** Handles all CRUD operations in one class.
- **Router:** Creates the corresponding URLs automatically (/books/, /books/<id>/).

This makes API development **faster and cleaner**.



66. Describe JWT Authentication using `django-rest-framework-simplejwt` with setup steps.. Easy Answer

JWT Authentication in DRF using `django-rest-framework-simplejwt`:

Purpose:

- JWT (JSON Web Token) is used for **stateless authentication** in APIs.
- Tokens contain user info and are sent with requests to authenticate users.

Setup Steps:

1. Install the package

```
bash

pip install django-rest-framework-simplejwt
```

2. Update `settings.py`

```
python

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ),
}
```

3. Configure URLs for obtaining and refreshing tokens

```
python

from django.urls import path
from rest_framework_simplejwt.views import TokenObtainPairView, TokenRefreshView

urlpatterns = [
    path('api/token/', TokenObtainPairView.as_view(), name='token_obtain_pair'),
    path('api/token/refresh/', TokenRefreshView.as_view(), name='token_refresh'),
]
```

4. Use the token in requests

- Obtain token by POSTing credentials to `/api/token/`.
- Include token in request headers:

```
makefile
```

```
Authorization: Bearer <your_token_here>
```

Summary:

- Provides **secure, stateless authentication** for APIs.
- Supports **access and refresh tokens** for better security.



67. Explain the role of Django templates in the MVT architecture with an example. Easy Answer

Role of Django Templates in MVT Architecture: ✓

Purpose:

- Templates handle the **presentation layer** in Django.
- They define **how data is displayed** to the user (HTML, CSS, JavaScript).
- Templates separate **UI** from business logic in Views and Models.

Example: Display a list of books

```
html

<!-- templates/book_list.html -->
<h1>Book List</h1>
<ul>
  {% for book in books %}
    <li>{{ book.title }} by {{ book.author }}</li>
  {% endfor %}
</ul>
```

Usage in View:

```
python

from django.shortcuts import render
from .models import Book

def book_list(request):
    books = Book.objects.all()
    return render(request, 'book_list.html', {'books': books})
```

Summary:

- **Model** → Provides data.
- **View** → Processes data and sends it to template.

- **Template** → Renders HTML to display data to users.

Templates make Django applications **dynamic and user-friendly**.



68. Describe the process of creating a custom user model in Django.. Easy Answer

Creating a Custom User Model in Django:

Purpose:

- Allows you to **extend or modify** the default Django **User** model (e.g., add fields like **phone_number**).

Steps:

1. **Create a custom user model** by inheriting **AbstractUser** or **AbstractBaseUser**

```
python

from django.contrib.auth.models import AbstractUser
from django.db import models

class CustomUser(AbstractUser):
    phone_number = models.CharField(max_length=15, blank=True)
```

2. **Update settings.py**

```
python

AUTH_USER_MODEL = 'myapp.CustomUser'
```

3. **Create a custom user manager** (if using **AbstractBaseUser**) to handle user creation.

4. **Make migrations and migrate**

```
bash

python manage.py makemigrations
python manage.py migrate
```

5. **Use the custom user model** throughout your project for authentication.

Summary:

- Extends default User functionality.
- Must set **AUTH_USER_MODEL** **before running initial migrations**.
- Ensures consistent user management across the project.



69. Explain how to handle file validation in Django forms.. Easy Answer

Handling File Validation in Django Forms:

Purpose:

- Ensure uploaded files meet certain criteria like **file type, size, or content** before saving.

Steps:

1. Create a form with FileField or ImageField

```
python

from django import forms
from .models import Document

class DocumentForm(forms.ModelForm):
    class Meta:
        model = Document
        fields = ['title', 'file']
```

2. Add custom validation method

```
python

def clean_file(self):
    file = self.cleaned_data.get('file')
    if file:
        # Check file size (example: 5MB max)
        if file.size > 5*1024*1024:
            raise forms.ValidationError("File size should not exceed 5MB.")
        # Check file extension
        if not file.name.endswith('.pdf'):
            raise forms.ValidationError("Only PDF files are allowed.")
    return file
```

3. Use the form in views

- Form will **raise validation errors** if the file does not meet criteria.

Summary:

- `clean_<fieldname>()` method allows **custom validation** for each file field.
- Ensures **security and data integrity** for uploaded files.



70. Write token-based authentication vs session-based authentication in Django REST Framework. Easy Answer

Token-based Authentication vs Session-based Authentication in Django REST Framework:

Feature	Token-based Authentication	Session-based Authentication
How it works	Client sends a token (e.g., JWT) with each request.	Server creates a session and stores it; client uses session cookie .
Stateless / Stateful	Stateless – no server-side session storage.	Stateful – server stores session data.

Feature	Token-based Authentication	Session-based Authentication
Best Use Case	REST APIs, mobile apps, microservices.	Web applications with browser-based login.
Security	Token can be expired, signed, and stored client-side.	Relies on server-side session security.
Example	Authorization: Bearer <token> header	Session cookie automatically sent with requests

Summary:

- **Token-based** → Lightweight, scalable, suitable for APIs.
- **Session-based** → Works well for traditional web apps with browsers.



71. Write the steps to implement API versioning in Django REST Framework.. Easy Answer

Steps to Implement API Versioning in Django REST Framework (DRF):

Purpose:

- API versioning helps **maintain multiple versions** of an API without breaking existing clients.

Steps:

1. **Update settings.py** to specify default versioning:

```
python

REST_FRAMEWORK = {
    'DEFAULT_VERSIONING_CLASS': 'rest_framework.versioning.URLPathVersioning',
    'DEFAULT_VERSION': 'v1',
}
```

2. **Define URLs with version prefix**

```
python

from django.urls import path, include
from rest_framework.routers import DefaultRouter
from .views import BookViewSet

router = DefaultRouter()
router.register(r'books', BookViewSet)

urlpatterns = [
    path('api/v1/', include(router.urls)), # Version 1
    path('api/v2/', include(router.urls)), # Version 2 (can point to a new viewset)
]
```

3. **Access version in views (optional)**

```
python

from rest_framework.views import APIView
from rest_framework.response import Response
```

```
class VersionedView(APIView):
    def get(self, request, *args, **kwargs):
        return Response({'version': request.version})
```

Summary:

- Versioning can be done via **URL, query parameter, or header**.
- Helps manage **breaking changes** while supporting older clients.



72. Discuss the structure of an Odoo module with examples of essential files. Easy Answer

Structure of an Odoo Module: ✓

An Odoo module is a **self-contained unit** that adds features to Odoo.

Essential Files and Folders:

1. `__manifest__.py`

- Contains **module metadata** like name, version, dependencies, and data files.

```
python

{
    'name': 'Library Management',
    'version': '1.0',
    'depends': ['base'],
    'data': ['views/library_book_views.xml'],
}
```

2. `__init__.py`

- Initializes the module and imports **Python files** like models and controllers.

```
python

from . import models
from . import controllers
```

3. `models/` folder

- Contains Python files defining **database models**.

```
python

# models/library_book.py
from odoo import models, fields

class Book(models.Model):
    _name = 'library.book'
    name = fields.Char(string='Title')
    author_id = fields.Many2one('res.partner', string='Author')
```

4. `views/` folder

- Contains XML files defining **forms, tree, kanban, and menu items**.

```
xml

<!-- views/library_book_views.xml -->
<record id="view_book_tree" model="ir.ui.view">
  <field name="name">library.book.tree</field>
  <field name="model">library.book</field>
  <field name="arch" type="xml">
    <tree>
      <field name="name"/>
      <field name="author_id"/>
    </tree>
  </field>
</record>
```

5. Optional folders/files:

- **controllers/** → Python files for HTTP routes.
- **security/** → Access control and record rules.
- **data/** → Demo or initial data.

Summary:

- **__manifest__.py** → Module info
- **__init__.py** → Initialize module
- **models/** → Database models
- **views/** → UI and menus
- Additional folders for security, controllers, and data as needed.



73. Explain relational fields in Odoo (Many2one, One2many, Many2many) with practical examples.. Easy Answer

Relational Fields in Odoo:

Relational fields define **relationships between models**.

1. Many2one

- **Many records** → **One record** (Foreign Key).
- **Example:** Many books belong to one author.

```
python

author_id = fields.Many2one('res.partner', string='Author')
```

2. One2many

- **One record** → **Many records** (Reverse of Many2one).

- **Example:** One author has many books.

```
python
```

```
book_ids = fields.One2many('library.book', 'author_id', string='Books')
```

3. Many2many

- **Many records** ↔ **Many records** (Bidirectional).
- **Example:** Books can belong to multiple categories, and categories can have multiple books.

```
python
```

```
category_ids = fields.Many2many('library.category', string='Categories')
```

Summary Table:

Field Type	Relationship	Example Use Case
Many2one	Many → One	Book → Author
One2many	One → Many	Author → Books
Many2many	Many ↔ Many	Book ↔ Categories

- Relational fields help **link models** and display related data in views.



74. Explain relational fields in Odoo (Many2one, One2many, Many2many) with practical examples.. Easy Answer

Relational Fields in Odoo:

Relational fields define **relationships between different models** in Odoo.

1. Many2one

- **Many records** → **One record** (like a Foreign Key).
- **Example:** Many books belong to one author.

```
python
```

```
author_id = fields.Many2one('res.partner', string='Author')
```

2. One2many

- **One record** → **Many records** (reverse of Many2one).
- **Example:** One author has many books.

```
python
```

```
book_ids = fields.One2many('library.book', 'author_id', string='Books')
```

3. Many2many

- **Many records ↔ Many records** (bidirectional relationship).
- **Example:** Books can belong to multiple categories, and categories can have multiple books.

```
python
```

```
category_ids = fields.Many2many('library.category', string='Categories')
```

Summary:

- **Many2one:** Book → Author
- **One2many:** Author → Books
- **Many2many:** Book ↔ Categories

These fields are used to **link models and display related data** in Odoo views.



75. Explain the process of creating a custom report in Odoo using QWeb templates.. Easy Answer

Creating a Custom Report in Odoo using QWeb Templates:

Purpose:

- QWeb templates are used to **design HTML/PDF reports** with dynamic data from Odoo models.

Steps:

1. Create a QWeb template in views/ folder

```
xml
```

```
<template id="report_book_list">
  <t t-call="web.html_container">
    <t t-foreach="docs" t-as="book">
      <p><t t-esc="book.name"/> by <t t-esc="book.author_id.name"/></p>
    </t>
  </t>
</template>
```

2. Define the report action in XML

```
xml
```

```
<record id="action_report_book" model="ir.actions.report">
  <field name="name">Book List Report</field>
  <field name="model">library.book</field>
```

```
<field name="report_type">qweb-pdf</field>
<field name="report_name">your_module.report_book_list</field>
<field name="print_report_name">'Book_List_Report'</field>
</record>
```

3. **Add a menu item or button** to trigger the report (optional)

4. **Use t-foreach and t-esc** in the template

- **t-foreach** → Loop through records
- **t-esc** → Display field values safely

Summary:

- **QWeb template** → Defines report layout
- **Report action** → Links template to a model and report type
- Can generate **PDF or HTML reports** dynamically from Odoo data.